CSC1110/CSC1122: Natural Language Technologies/ Foundations of NLP

Lecture 3: Text Classification

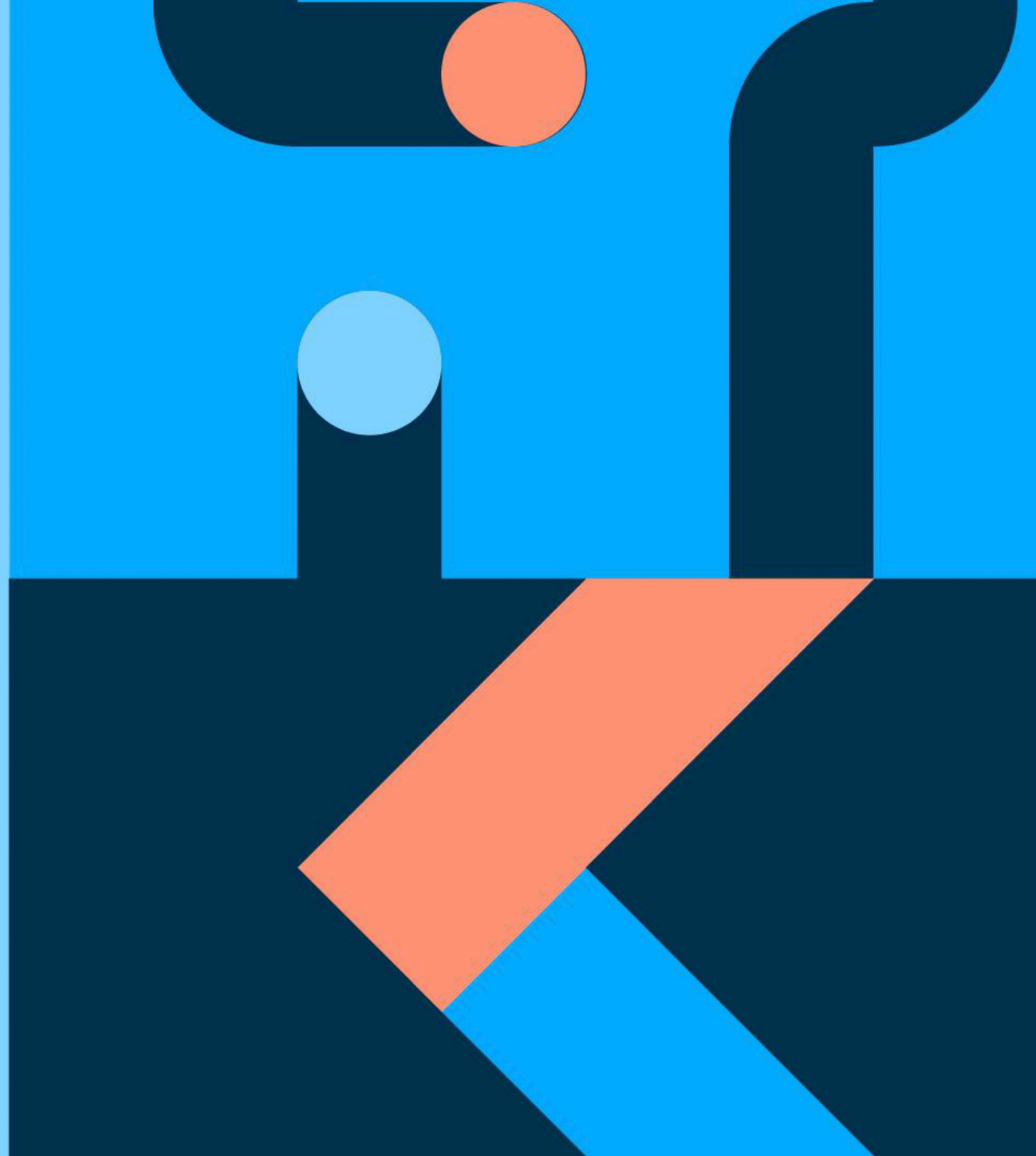
Dr. Ellen Rushe
Assistant Professor
School of Computing
Dublin City University

L2.24 (McNulty Building)
ellen.rushe@dcu.ie

DCU
Ollscoil Chathair
Bhaile Átha Cliath
Dublin City University

The contents of these slides are based on Chapter 3 of Jurafsky and Martin *Speech and Language Processing (3rd Edition)*

With special thanks to Prof. Jennifer Foster for a wealth of past slides!



An Introduction to Text Classification

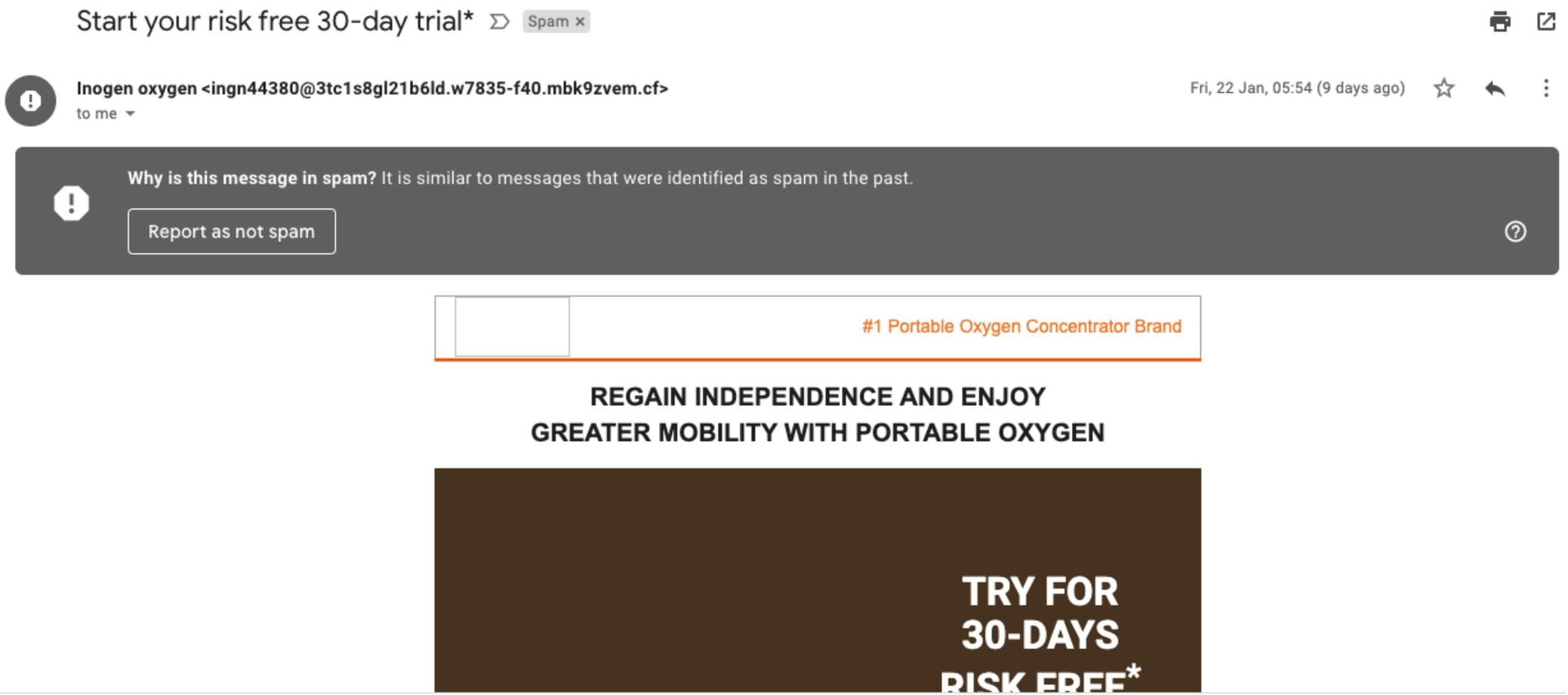
Text Classification Applications

- Sentiment analysis
- Spam detection
- Authorship identification
- Language Identification
- Topic Classification
- …

Positive or negative review?

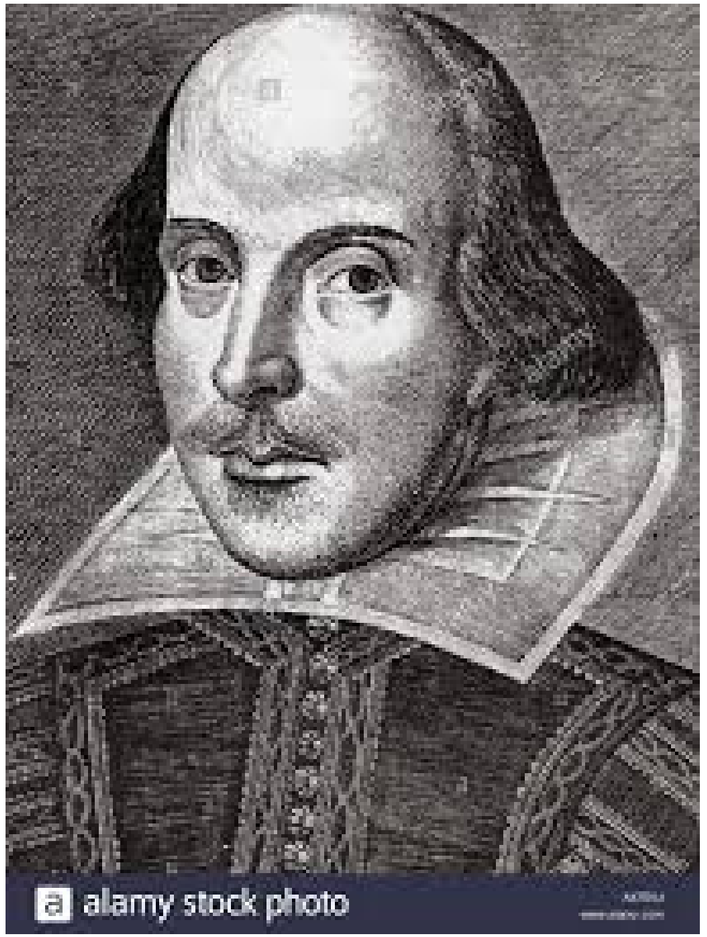
...zany characters and richly applied satire, and some great plot twists
...awesome caramel sauce and sweet toasty almonds. I love this place!
It was pathetic. The worst part about it was the boxing scenes...
...awful pizza and ridiculously overpriced...
Raised by Wolves is a joyless, claustrophobic and chilling treat

Is this spam?

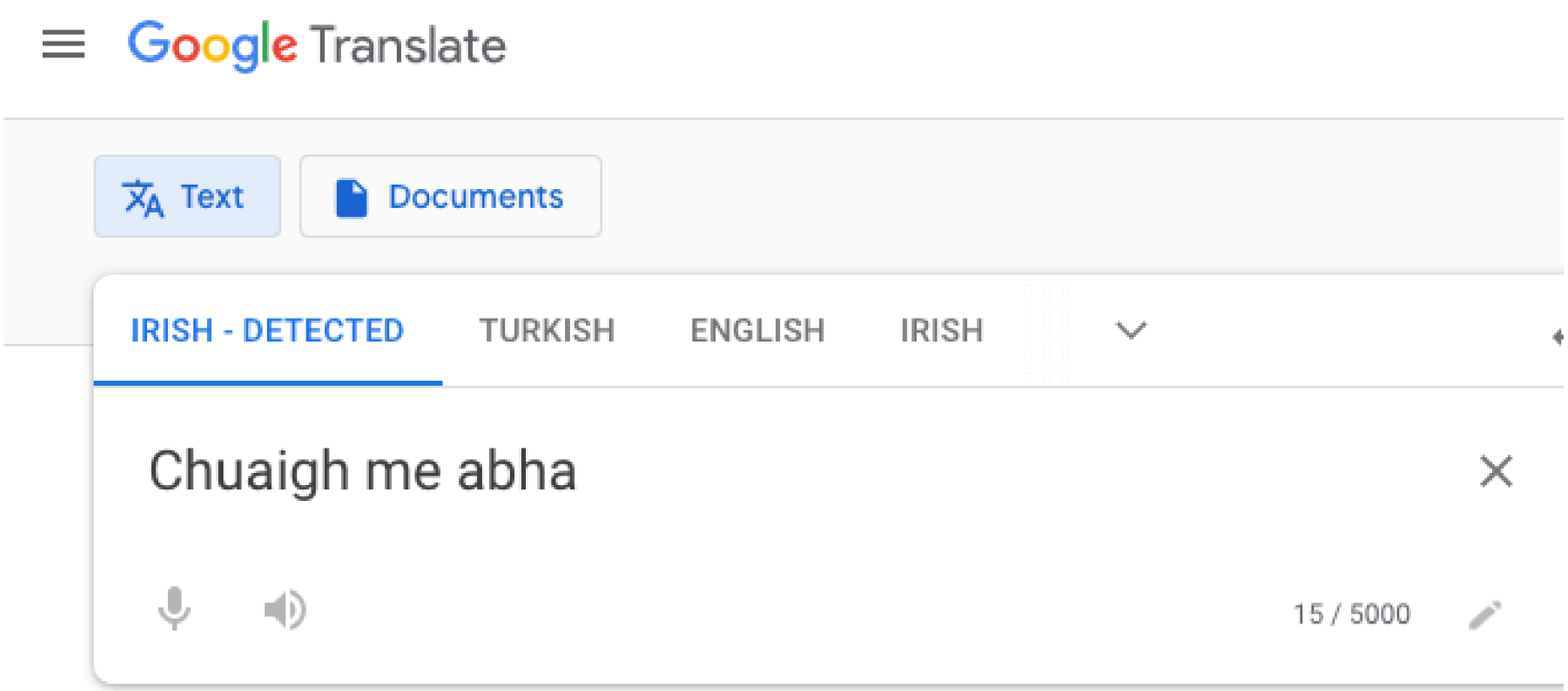

Start your risk free 30-day trial*
Spam ×
Inogen oxygen <ingn44380@3tc1s8gl21b6ld.w7835-f40.mbk9zvem.cf>
to me
Fri, 22 Jan, 05:54 (9 days ago)
Why is this message in spam? It is similar to messages that were identified as spam in the past.
Report as not spam
#1 Portable Oxygen Concentrator Brand
REGAIN INDEPENDENCE AND ENJOY
GREATER MOBILITY WITH PORTABLE OXYGEN
TRY FOR
30-DAYS

Authorship attribution

Did Shakespeare really write the complete works of Shakespeare?



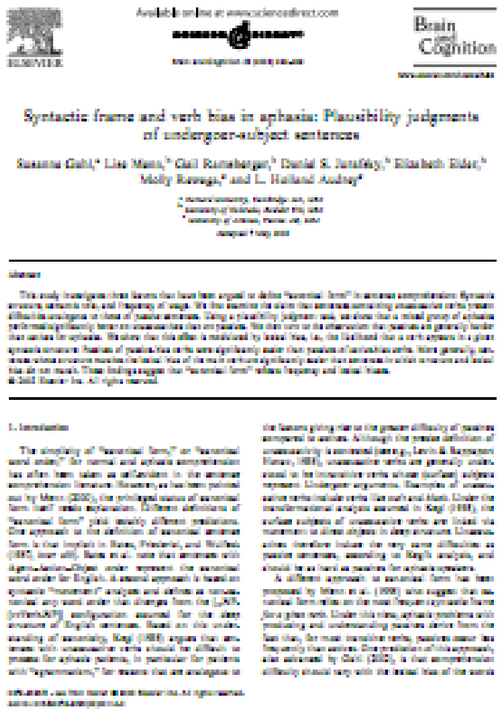
Language Identification



What is the subject of this medical article?

Medical Subject Headers (MeSH)Subject Category Hierarchy

MEDLINE Article



Syntactic frame and verb bias in aphasia: Plausibility judgments of undergoer-subject sentences

?

- Antogonists and Inhibitors
- Blood Supply
- Chemistry
- Drug Therapy
- Embryology
- Epidemiology
- …

Text Classification: Definition

- *Input*:
  - a document d
  - a fixed set of classes $C = \{c_1, c_2, \ldots, c_J\}$
- *Output*: a predicted class $c \in C$

Hand-coded rules

- Rules based on combinations of words or other features
  - spam: black-list-address
  - “dollars” AND “you have been selected” etc.
- Accuracy can be high
  - If rules carefully refined by expert
- But building and maintaining these rules is expensive

Supervised Machine Learning

- *Input to training*:
  - A training set of *m* hand-labelled documents:
    $(d_1,c_1),\ldots,(d_m,c_m)$

    where for each (d_i, c_i) in the training set,

- *Output of training*:
  - a classifier (function) which maps a document to a class, $y : d \rightarrow c$

Supervised Machine Learning

- Any kind of classifier
  - Naïve Bayes
  - Logistic regression
  - Neural networks
  - k-Nearest Neighbour
  - …

The Naïve Bayes Classifier

Bayes' Rule

For a document *d* and a class *c*

$$P(c \mid d) = \frac{P(d \mid c)P(c)}{P(d)}$$

Naive Bayes Classifier (I)

$$c_{MAP} = \underset{c \in C}{\operatorname{argmax}} P(c \mid d)$$

MAP is "maximum a posteriori" = most likely class

$$= \underset{c \in C}{\operatorname{argmax}} \frac{P(d \mid c)P(c)}{P(d)}$$

Bayes Rule

$$= \underset{c \in C}{\operatorname{argmax}} P(d \mid c)P(c)$$

Dropping the denominator

Naïve Bayes Classifier (II)

"Likelihood" "Prior"

$$c_{MAP} = \underset{c \in C}{\operatorname{argmax}} P(d \mid c) P(c)$$

$$= \underset{c \in C}{\operatorname{argmax}} P(x_1, x_2, \ldots, x_n \mid c) P(c)$$

Document d represented as features x1..xn

Naïve Bayes Classifier (III)

$$c_{MAP} = \underset{c \in C}{\operatorname{argmax}} P(x_1, x_2, \ldots, x_n \mid c) P(c)$$

Could only be estimated if a very, very large number of training examples was available.

How often does this class occur?

We can just count the relative frequencies in a corpus

Naïve Bayes Classifier (IV)

$$c_{MAP} = \underset{c \in C}{\operatorname{argmax}} P(x_1, x_2, \ldots, x_n \mid c) P(c)$$

Assume the features *x* are **independent**

$$P(x_1, \ldots, x_n \mid c) = P(x_1 \mid c) \bullet P(x_2 \mid c) \bullet P(x_3 \mid c) \bullet \ldots \bullet P(x_n \mid c)$$

Naïve Bayes Classifier (V)

Given a document, *d*, with *n* tokens w_1 w_2 … *wn*

$$C_{NB} = \underset{c_j \in C}{\operatorname{argmax}} p(cj) \prod_{i=1}^{n} p(wi|cj)$$

Log space

Instead of this

$$C_{NB} = \underset{c_j \in C}{\operatorname{argmax}} p(c_j) \prod_{i=1}^{n} p(w_i|c_j)$$

we use this

$$C_{NB} = \underset{c_j \in C}{\operatorname{argmax}} [\log p(c_j) + \sum_{i=1}^{n} \log p(w_i|c_j)]$$

This is ok since log doesn't change the ranking of the classes (class with highest prob still has highest log prob)

Training the Naïve Bayes Classifier

Learning a Naive Bayes Model

Maximum likelihood estimation

$$\hat{P}(c_j) = \frac{doccount(C = c_j)}{N_{doc}}$$

$$\hat{P}(w_i \mid c_j) = \frac{count(w_i, c_j)}{\sum_{w \in V} count(w, c_j)}$$

Parameter estimation

$$\hat{P}(w_i \mid c_j) = \frac{count(w_i, c_j)}{\sum_{w \in V} count(w, c_j)}$$

fraction of times word *wi* appears among all words in documents of class *cj*

- Create “mega-document” for class *j* by concatenating all docs with class j
- Use frequency of *w* in mega-document

Zero probabilities (again)

"Raised by Wolves is a joyless, claustrophobic and chilling treat"

- What if we have seen some negative training documents but no positive training documents with the word ***joyless***?
  p(joyless|positive) = 0

- Zero probabilities propagate

$$c_{MAP} = \text{argmax}_c \hat{P}(c) \prod_i \hat{P}(x_i \mid c)$$

Laplace (add-1) smoothing (again)

$$\hat{P}(w_i \mid c) = \frac{count(w_i, c)}{\sum_{w \in V} \left(count(w, c) \right)}$$

$$= \frac{count(w_i, c) + 1}{\left(\sum_{w \in V} count(w, c) \right) + |V|}$$

Add-α smoothing

- Instead of 1 we can add the value α
- The value of α is empirically determined.

Unknown words

- What about unknown words
  - that appear in our test data
  - but not in our training data or vocab

- We **ignore** them
  - Remove them from the test document!
  - Pretend they weren't there!
  - Don't include any probability for them at all.

Stop words

- Some systems ignore another class of words:
- **Stop words:** very frequent words like *the* and *a*.
  - Sort the whole vocabulary by frequency in the training, call the top 10 or 50 words the **stopword list**.
  - Now we remove all stop words from the training and test sets as if they were never there.
- But in many text classification applications, removing stop words doesn't help that much...

Training Naïve Bayes

1. From training corpus, extract *Vocabulary*

2. Calculate $P(cj)$ terms
   - For each *cj* in *C* do
     docsj: all docs with class =*cj*

3. Calculate $P(wk \mid cj)$ terms
   - *Textj* : megadoc containing all *docsj*
   - For each word *wk* in *Vocabulary*
     nk: # of occurrences of *wk* in *Textj*

$$P(c_j) \leftarrow \frac{|docs_j|}{|\text{total \# documents}|}$$

$$P(w_k \mid c_j) \leftarrow \frac{n_k + \alpha}{n + \alpha |Vocabulary|}$$

A worked example

A worked sentiment example

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

A worked sentiment example

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

Prior from training:

P(−) = 3/5

P(+) = 2/5

Drop "with"

Likelihoods from training:

$$P(\text{“predictable”}|-) = \frac{1+1}{14+20} \qquad P(\text{“predictable”}|+) = \frac{0+1}{9+20}$$

$$P(\text{“no”}|-) = \frac{1+1}{14+20} \qquad P(\text{“no”}|+) = \frac{0+1}{9+20}$$

$$P(\text{“fun”}|-) = \frac{0+1}{14+20} \qquad P(\text{“fun”}|+) = \frac{1+1}{9+20}$$

Scoring the test set:

$$P(-)P(S|-) = \frac{3}{5} \times \frac{2 \times 2 \times 1}{34^3} = 6.1 \times 10^{-5}$$

$$P(+)P(S|+) = \frac{2}{5} \times \frac{1 \times 1 \times 2}{29^3} = 3.2 \times 10^{-5}$$

More on sentiment analysis

Optimizing for sentiment analysis

1. Binarized Naïve Bayes
2. Handling Negation
3. Sentiment Lexicons

Binarized Naïve Bayes

For tasks like sentiment, word **occurrence** is more important than word **frequency**.

- The occurrence of the word *fantastic* tells us a lot
- The fact that it occurs 5 times may not tell us much more.

Therefore

- Clip our (within-document) word counts at 1
- Remove duplicates from our training and test documents

Dealing with Negation

- *I really like this movie -> I really **don't** like this movie*

Negation changes the meaning of "like" to negative.
Negation can also change negative to positive-ish

- ***Don't** dismiss this film*
- ***Doesn't** let us get bored*

Dealing with Negation

Das, Sanjiv and Mike Chen. 2001. Yahoo! for Amazon: Extracting market sentiment from stock message boards. In Proceedings of the Asia Pacific Finance Association Annual Conference (APFA).
Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan. 2002. Thumbs up? Sentiment Classification using Machine Learning Techniques. EMNLP-2002, 79—86.

Simple baseline method:
Add NOT_ to every word between negation and following punctuation:

didn't like this movie , but I

didn't NOT_like NOT_this NOT_movie but I

Sentiment Classification: Lexicons

- Sometimes we don't have enough labelled training data
- In that case, we can make use of pre-built word lists
- Called **lexicons**
- There are various publicly available lexicons:
  - MPQA (https://mpqa.cs.pitt.edu/lexicons/subj_lexicon/)
  - NRC Emotion Lexicon (https://saifmohammad.com/WebPages/NRC-Emotion-Lexicon.htm)
  - Bing Liu's sentiment lexicon (https://juliasilge.github.io/tidytext/reference/sentiments.html)
  - SentiWordNet (https://github.com/aesuli/SentiWordNet)

Lexicons in Sentiment Classification

Add a feature that gets a count whenever a word from the lexicon occurs in the document

Example

Raised by Wolves is a joyless, claustrophobic and chilling treat

Imagine our sentiment lexicon contains “joyless”, “claustrophobic” and “chilling” and “treat” with the first three labelled as negative and the last as positive.

positive feature gets the value 1 and negative feature gets the value 3

Can help when training data is sparse or not representative of the test set

Naïve Bayes: points to note

Other types of features

Spam Detection

- Mentions millions of (dollar) ((dollar) NN,NNN,NNN.NN)
- From: starts with many numbers
- Subject is all capitals
- HTML has a low ratio of text to image area
- "One hundred percent guaranteed"
- Claims you can be removed from the list

Language identification

- Character n-grams, e.g. ‘ing’, will have high frequency count for English, ‘ich’ for German

Naïve Bayes as a Language Model

- Assigning each word: P(word | c)
- Assigning each sentence: P(s|c)=Π P(word|c)

Class *pos*

0.1 I
0.1 love
0.01 this
0.05 fun
0.1 film
…

I	love	this	fun	film
0.1	0.1	.05	0.01	0.1

P(s | pos) = 0.0000005

Naïve Bayes as a Language Model

- Which class assigns the higher probability to s?

Model pos	
0.1	I
0.1	love
0.01	this
0.05	fun
0.1	film

Model neg	
0.2	I
0.001	love
0.01	this
0.005	fun
0.1	film

I	love	this	fun	film
0.1	0.1	0.01	0.05	0.1
0.2	0.001	0.01	0.005	0.1

$$P(s|pos) > P(s|neg)$$

Evaluation

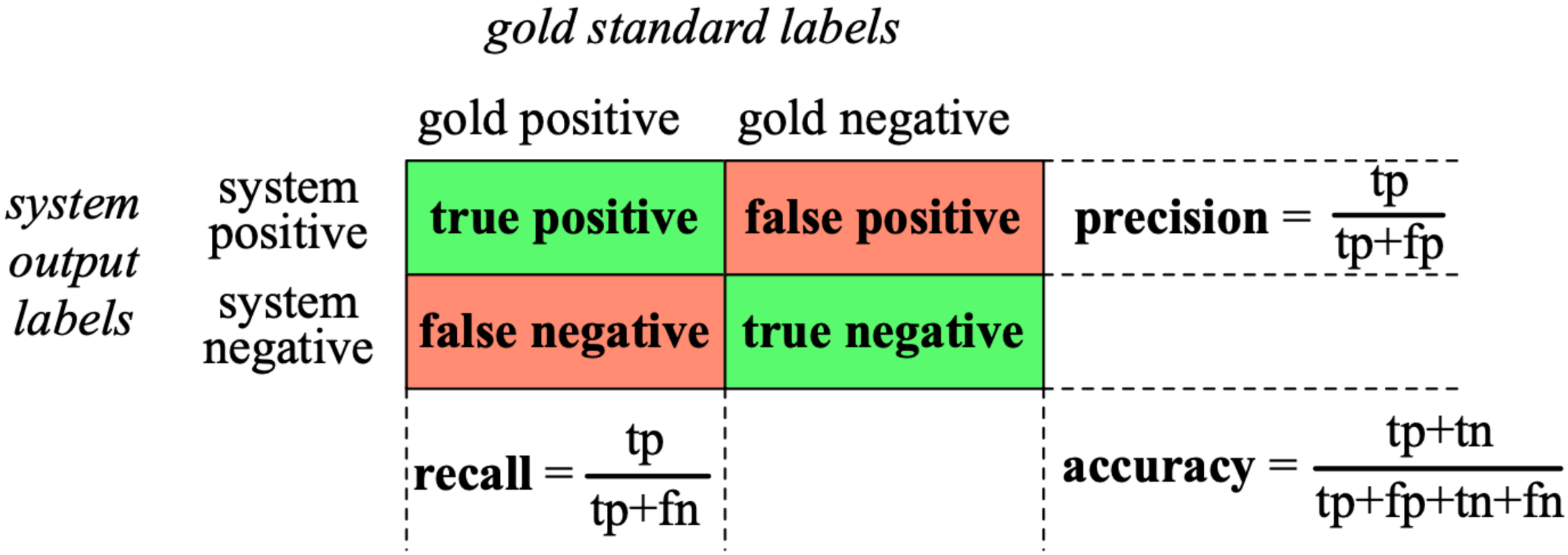
Accuracy

- Accuracy of a classifier is

$$\frac{\#CorrectPredictions}{\#TotalPredictions}$$

- Very common metric

The 2-by-2 confusion matrix



Precision and recall

- Imagine you're the CEO of Delicious Pie Company
- You want to know what people are saying about your pies
- So you build a "Delicious Pie" tweet detector
  - Positive class: tweets about Delicious Pie Co
  - Negative class: all other tweets

Precision and recall

- Imagine we saw 1 million tweets
  - 100 of them talked about Delicious Pie Co.
  - 999,900 talked about something else
- We could build a dumb classifier that just labels every tweet "not about pie"
  - It would get 99.99% accuracy!!! Wow!!!!
  - But useless! Doesn't return the comments we are looking for!
  - For this kind of problem we use **precision** and **recall** instead

Evaluation: Precision

- % of items the system detected (i.e. items the system labeled as positive) that are in fact positive (according to the human gold labels)

$$\textbf{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

Evaluation: Recall

- % of items actually present in the input that were correctly identified by the system.

$$\textbf{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

Why Precision and Recall

- Our dumb pie-classifier
  - Just label nothing as "about pie"
    Accuracy=99.99%

    but
    Recall = Precision = 0
  - (it doesn't get any of the 100 Pie tweets)
    Precision and recall, unlike accuracy, emphasize true positives:
  - finding the things that we are supposed to be looking for.

A combined measure: F

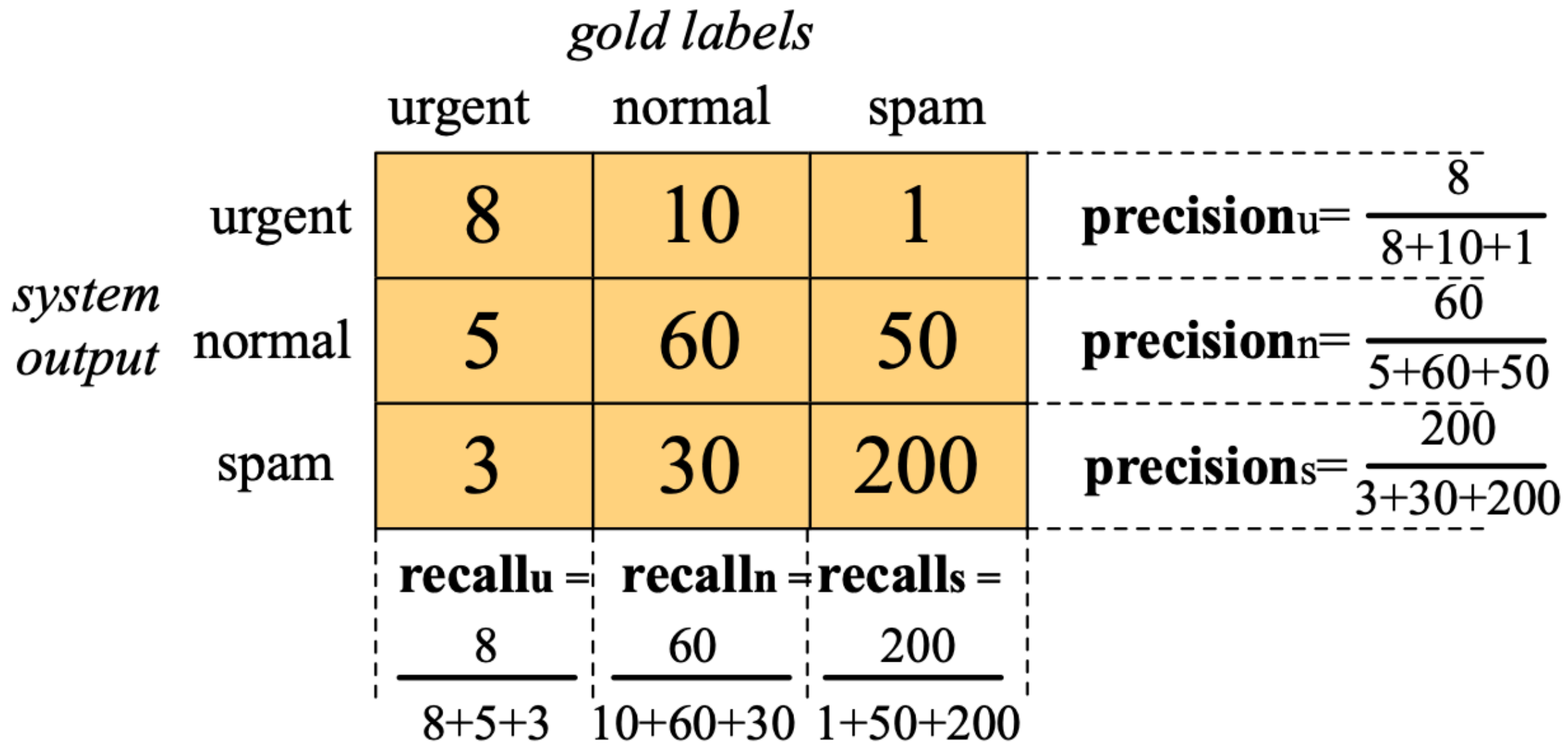
- F measure: a single number that combines P and R:

$$F_\beta = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}$$

- We almost always use balanced F_1 (i.e., $\beta = 1$)

$$\mathrm{F}_1 = \frac{2PR}{P + R}$$

Confusion Matrix for 3-class classification



Combining P/R from >2 classes

- **Macroaveraging:**
  - compute the performance for each class, and then average over classes
- **Microaveraging**:
  - collect decisions for all classes into one confusion matrix
  - compute precision and recall from that table.

Macroaveraging and Microaveraging

Class 1: Urgent

	true urgent	true not
system urgent	8	11
system not	8	340

$$\text{precision} = \frac{8}{8+11} = .42$$

Class 2: Normal

	true normal	true not
system normal	60	55
system not	40	212

$$\text{precision} = \frac{60}{60+55} = .52$$

Class 3: Spam

	true spam	true not
system spam	200	33
system not	51	83

$$\text{precision} = \frac{200}{200+33} = .86$$

$$\text{macroaverage precision} = \frac{.42+.52+.86}{3} = \mathbf{.60}$$

Pooled

	true yes	true no
system yes	268	99
system no	99	635

$$\text{microaverage precision} = \frac{268}{268+99} = \mathbf{.73}$$

Development Test Sets ("Devsets")

Training set	Development Test Set	Test Set

- **Train** on training set, **tune** on devset, **report** on testset
- This avoids overfitting ('tuning to the test set')
- More conservative estimate of performance

Cross-validation: multiple splits

- Pool results over splits, Compute pooled dev performance